



Université de Versailles Saint-Quentin-en-Yvelines  
Master 1 CHPS – AISE 2025/2026

---

# Rapport de Projet : Bibliothèque de Parallélisme EXPRESSO

---

*Auteur:*

Jianye SHI

*Enseignant:*

Marek Felsoci

January 6, 2026

## Sommaire

|          |                                                       |          |
|----------|-------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>3</b> |
| <b>2</b> | <b>Implémentation de la bibliothèque EXPRESSO</b>     | <b>3</b> |
| 2.1      | Architecture du Moteur d'Exécution . . . . .          | 3        |
| 2.2      | Politiques d'Ordonnancement . . . . .                 | 3        |
| <b>3</b> | <b>Parallélisation des Applications</b>               | <b>4</b> |
| 3.1      | Produit Scalaire (dot.c) . . . . .                    | 4        |
| 3.2      | Traitement de Liste (list.c) . . . . .                | 4        |
| 3.3      | Produit Élément par Élément (vecmul.c) . . . . .      | 4        |
| 3.4      | Produit Matriciel (matmul.c) . . . . .                | 5        |
| <b>4</b> | <b>Analyse des Performances</b>                       | <b>5</b> |
| 4.1      | Résultats Expérimentaux . . . . .                     | 5        |
| 4.2      | Comparaison des Politiques d'Ordonnancement . . . . . | 5        |
| <b>5</b> | <b>Conclusion</b>                                     | <b>6</b> |

## 1 Introduction

Ce rapport présente la réalisation du projet **EXPRESSO** (*Experimental parallel resource scheduler and organizer*), une bibliothèque de parallélisme en mémoire partagée basée sur le paradigme *Fork-Join*. L'objectif est double : implémenter une interface de programmation parallèle de haut niveau similaire à OpenMP, et utiliser cette bibliothèque pour paralléliser un ensemble d'applications séquentielles de référence (`dot`, `list`, `vecmul`, `matmul`).

Le projet s'inscrit dans le cadre du module "Architecture interne des systèmes d'exploitation" (AISE) et vise à approfondir la compréhension des mécanismes bas niveau tels que la gestion des threads POSIX, la synchronisation, et l'ordonnancement de tâches.

## 2 Implémentation de la bibliothèque EXPRESSO

La bibliothèque a été développée en langage C, en respectant la norme C99 et sans modifier le `Makefile` fourni, conformément aux contraintes du sujet.

### 2.1 Architecture du Moteur d'Exécution

EXPRESSO repose sur un modèle de **Thread Pool** (pool de threads). Lors de l'initialisation (`expresso_initialize`), une équipe de threads travailleurs est créée. Cette équipe reste active pendant toute la durée de vie du programme, évitant ainsi le coût prohibitif de la création et destruction répétée de threads pour chaque tâche.

#### 2.1.1 Synchronisation par Barrières

Pour orchestrer le modèle Fork-Join, nous avons préféré utiliser des **barrières POSIX** (`pthread_barrier_t`) plutôt que des variables de condition. Cette approche nous a semblé plus intuitive pour garantir que tous les threads démarrent et s'arrêtent au même moment, évitant ainsi des erreurs de synchronisation complexes.

- **Barrière de départ** : Synchronise le démarrage des calculs.
- **Barrière de fin** : Assure que tout le monde a fini avant de rendre la main.

#### 2.1.2 Opérations Atomiques et Conformité C99

L'utilisation d'un Mutex global pour les statistiques (`expresso_stats`) aurait trop ralenti l'exécution. Nous avons donc choisi d'utiliser les **fonctions intégrées GCC** pour l'incrément atomique. Comme le projet impose le standard **C99**, nous ne pouvions pas utiliser `<stdatomic.h>` (C11), ce qui nous a conduits à cette solution :

```
// Exemple d'incrément atomique conforme C99
__atomic_fetch_add(&sys.tasks_executed[id], 1, __ATOMIC_RELAXED);
```

Cette solution allie performance et respect strict des contraintes de compilation.

*Note sur la gestion de la file d'attente* : Nous avons opté pour une file d'attente protégée par un Mutex global. Bien que cette approche puisse théoriquement introduire un goulot d'étranglement avec un très grand nombre de tâches, elle reste simple et performante pour des tâches de granularité moyenne ("coarse-grained") comme celles de nos benchmarks, assurant la correction du programme sans complexité excessive.

### 2.2 Politiques d'Ordonnancement

La bibliothèque supporte trois politiques d'ordonnancement, sélectionnables dynamiquement :

1. **SCHEDULE\_DYNAMIC** (Défaut) : Les tâches sont placées dans une file d'attente globale protégée par un Mutex. Les travailleurs récupèrent les tâches une par une ("First Come, First Served"). Cette méthode favorise un équilibrage de charge mais introduit une contention sur le verrou de la file.
2. **SCHEDULE\_STATIC** : Les tâches sont attribuées aux threads de manière cyclique (*Round-Robin*) au moment de la soumission.

$$\text{Thread ID} = \text{Task ID} \% \text{Nombre de Travailleurs}$$

Cette politique peut réduire la contention au moment de l'exécution mais peut créer un déséquilibre de charge si les tâches ont des durées inégales.

3. **SCHEDULE\_BALANCED** : Cette politique utilise les poids associés aux tâches.

`expresso_weighted_task`

L'algorithme attribue chaque nouvelle tâche au travailleur ayant actuellement la charge cumulée (somme des poids) la plus faible. C'est une approche gloutonne effectuant une **répartition statique à la soumission** (basée sur l'estimation de charge) pour réduire le *makespan*, et non un mécanisme dynamique de vol de travail (*work stealing*).

### 3 Parallélisation des Applications

Nous avons parallélisé quatre applications de référence. Pour chacune, l'initialisation et l'allocation mémoire sont effectuées avant la section chronométrée, qui inclut l'exécution et la soumission des tâches. De plus, pour respecter la norme C99, toutes les fonctions de tâches et structures de données ont été définies au niveau global (hors du `main`), évitant les fonctions imbriquées (extension GNU).

#### 3.1 Produit Scalaire (`dot.c`)

**Stratégie** : Décomposition par blocs. Les vecteurs sont divisés en  $N$  blocs contigus. Chaque tâche calcule la somme partielle d'un bloc. Le résultat final est la somme des sommes partielles.

- *Données* : Structure contenant les pointeurs vers le début du bloc et la taille.
- *Performance* : Très bonne scalabilité car les tâches sont indépendantes et équilibrées.

#### 3.2 Traitement de Liste (`list.c`)

**Stratégie** : Une tâche par nœud. Le thread principal parcourt la liste chaînée et soumet une tâche pour chaque nœud (fonction `list_process_node`).

- *Défi* : Le coût de création de tâche peut être élevé par rapport au traitement d'un nœud si celui-ci est très rapide. Cependant, dans ce benchmark, `process_node` contient un `sleep`, rendant le surcoût de gestion négligeable face au gain du parallélisme.

#### 3.3 Produit Élément par Élément (`vecmul.c`)

**Stratégie** : Décomposition par blocs, similaire à `dot.c`. Chaque tâche traite une portion des vecteurs  $V1$  et  $V2$  et écrit le résultat directement dans  $V1$ .

- *Scheduling* : La politique par défaut est **SCHEDULE\_DYNAMIC**. Un passage explicite à **SCHEDULE\_STATIC** peut être pertinent si l'on souhaite imposer un découpage cyclique sur des coûts uniformes.

### 3.4 Produit Matriciel (matmul.c)

**Stratégie** : Décomposition par blocs de lignes. Pour calculer  $C = A \times B$ , nous divisons la matrice  $C$  (et  $A$ ) en groupes de lignes (`row_block`).

- *Poids des tâches* : Le coût calculatoire est proportionnel au nombre de lignes du bloc  $\times K \times M$ . Nous utilisons cette valeur comme poids pour la soumission via `expresso_weighted_task`. Ce poids est exploité par `SCHEDULE_BALANCED` si cette politique est sélectionnée.
- *Optimisation* : Cette approche favorise la localité spatiale pour la matrice  $C$  et  $A$ .

## 4 Analyse des Performances

Les tests ont été réalisés sur une machine de TP standard. Nous avons observé les tendances qualitatives suivantes, qui restent dépendantes des paramètres et de l’environnement d’exécution :

- **Overhead** : Pour de très petites charges, la version séquentielle reste plus rapide que la version parallèle à cause du surcoût de gestion des tâches et de synchronisation.
- **Scalabilité** : Sur des charges suffisantes (ex: `dot` avec grands vecteurs, `matmul`  $N \geq 512$ ), un *speedup* significatif est attendu, sans garantie de linéarité parfaite.
- **Impact du Scheduler** :
  - Sur des charges homogènes (`vecmul`, `matmul` régulier), `STATIC` et `DYNAMIC` peuvent offrir des performances comparables, selon la contention et la granularité des tâches.
  - Sur des charges hétérogènes (simulées ou `list` avec temps variables), `DYNAMIC` et `BALANCED` peuvent mieux utiliser les ressources en réduisant l’inactivité.

### 4.1 Résultats Expérimentaux

Nous avons mesuré les performances sur l’application `matmul` (multiplication de matrices  $512 \times 512$ , blocs de 64 lignes) sur une machine 8 cœurs. Les résultats ci-dessous illustrent un exemple de gain observé, dépendant des conditions d’exécution et des paramètres utilisés.

| Threads | Temps Séquentiel (Ref) | Temps Parallèle (Expresso) | Speedup |
|---------|------------------------|----------------------------|---------|
| 1       | 0.35 s                 | 0.35 s                     | 1.0x    |
| 2       | 0.35 s                 | 0.18 s                     | 1.9x    |
| 4       | 0.35 s                 | 0.10 s                     | 3.5x    |
| 8       | 0.35 s                 | 0.05 s                     | 7.4x    |

Table 1: Performances de `matmul` ( $N=512$ ) avec la politique `SCHEDULE_DYNAMIC`

On observe un gain de performance important ( $7.4\times$  pour 8 threads) dans cet exemple. Ce résultat est satisfaisant.

### 4.2 Comparaison des Politiques d’Ordonnancement

Nous avons comparé les trois politiques sur le benchmark `matmul` ( $N = 512$ , bloc=64, 32 threads disponibles, 8 tâches réelles).

| Politique | Temps d’exécution | Observation                                  |
|-----------|-------------------|----------------------------------------------|
| Dynamic   | 0.093 s           | Allocation dynamique, légère contention      |
| Static    | 0.088 s           | Allocation fixe, légèrement plus rapide      |
| Balanced  | 0.094 s           | Similaire au dynamique sur charges uniformes |

Table 2: Impact de l’ordonnancement sur `matmul` ( $N=512$ )

On constate que la politique **STATIC** offre ici les meilleures performances (environ 5% de gain), probablement grâce à une attribution déterministe qui minimise les conflits entre threads lors de la récupération des tâches, même si le verrou global reste présent. La politique **BALANCED** se comporte de manière similaire à **DYNAMIC** car les tâches ont ici des poids identiques (blocs de taille fixe), rendant l'équilibrage de charge "intelligent" peu pertinent pour ce cas homogène.

## 5 Conclusion

En conclusion, le développement d'EXPRESSO a mis en lumière l'importance des détails d'implémentation dans un runtime parallèle. Au-delà du respect des contraintes C99, le véritable défi a été de minimiser l'overhead de synchronisation. Les tests montrent que notre approche (Barrières + Atomiques) paie sur les calculs intensifs comme `matmul`, même si les limites se font sentir sur les tâches trop légères (`dot`). Ce projet valide donc qu'une gestion fine des ressources est indispensable pour obtenir un speedup réel.